Review_for_midterm

October 14, 2026

1 Review for Midterm

Try to solve these problems without running/writing code. Instead you should try to come up with your answers by tracing through the code yourself and reasoning about what the Python interpreter does.

Note: You will see similar questions on the Pen and Paper Midterm.

Note: This review is not exhaustive, you should study all materials in portfolios (1-5) and textbook chapters (1-7).

[ ]:
```
# 1. What gets printed by the following code?

x = 15
while x > 5:
    print(x,end = " ")
    x = x-1
```

[ ]:
```
# 2. What gets printed by the following code?
# while loop example
x = 15
while x > 5:
    x = x - 1
    print(x, end = " ")
```

[ ]:
```
# 3. What gets printed by the following code?

i = 2
while i < 6:
    print(i, end = " ")
```

[ ]:
```
# 4.        What does this snippet of code do?
valid = False
while not valid:
    x = int(input("Enter a number: "))

    valid = (x % 2 == 0 and x % 6 == 0)
```

[ ]:
```
# 5.        How can we translate this code into a while loop?
#        for i in range(n):
#             <body>
n = 5
for i in range(n):
    print(i)

# while loop
```

[ ]:
```
# 6. What is the value of x after this code runs?
x = -5
if x > 2:
    x = -3
    x = 1
else:
    x = 3
    x = 2
print(x)
```

[ ]:
```
# 7. What is the value of x after this code runs?
x = 2
if x >= 2:
    x = -3
    x = 1
else:
    x = 3
    x = 2
print(x)
```

[ ]:
```
# 8. Given the function definition for is_even below, what will be printed to␣
 ↪the screen when this code is run?
def is_even(x):
    if x % 2 == 0:
        return True
    else:
        return False

print(is_odd(1))
print(is_odd(2))
#print(is_odd(3.5))
#print(is_odd("abc"))
```

[ ]:
```
# 9. Consider the following function definition for is_odd. Can you rewrite the␣
 ↪function so that it only uses one line after "def is_odd(x):"? If so,␣
 ↪provide an example.
def is_even(x):
    if x % 2 == 0:
```

```
        return True
    else:
        return False
print(is_odd(2))
print(is_odd(45265271))
```

[ ]:
```
# 10. What gets printed when the following code runs?
grade = 98
if (grade >= 90):
    print("You got an A!")
if (grade >= 80):
    print("You got a B!")
else:
    print("You got another grade.")
```

[ ]:
```
# 11. What gets printed when the following code runs?
grade = 98
if (grade >= 90):
    print("You got an A!")
elif (grade >= 80):
    print("You got a B!")
else:
    print("You got another grade.")
```

12. Please fill in the blanks in the code below so the followin payscale is followed.

		experience (years)		
		0	1-2	3+
age	<18	15.25	16.00	16.00
	18+	15.25	16.25	17.50

[ ]:
```
experience = 1
age =  22
if experience == 0:
    wage = 15.25
elif (<1>):
    if (<2>):
        wage = 17.50
    else:
        wage = 16.25
else:
    wage = 16.0

print(wage)
```

[ ]:
```
# 13. What will the value of y be after this code runs?
x = 12
y = (x > 5) and ((not(x>12))or(x==5))
print(y)
```

[ ]:
```
# 14. What is the value of name2 after this code is run?

num1 = 1212
num2 = 666

temp = num1
num1 = num2
num2 = temp
```

[ ]:
```
# 15. What will this print

100DaysofSummer = 10
print(100DaysofSummer
```

[ ]:
```
# 16. What will this print

TaylorSwift = 10
print(TaylorSwift)
```

[ ]:
```
# 17. What will this print

for = 11
print(for)
```

[ ]:
```
# 18. What will this print

Ginny is So Cute = 10
print(Ginny is So Cute)
```

[ ]:
```
# 19. What will this print

_Dolly_Parton__________ = 10
print(_Dolly_Parton__________)
```

[ ]:
```
# 20. What will this print

georgiadoing@github.io = 10
print(georgiadoing@github.io)
```

[ ]:
```
# 21. What is the value of each variable after this code runs?
var1 = "Hello"
var2 = 2
```

```
var3 = "Re:World"
var4 = var2 + 10
var2 = 86

print(var1, var2, var3, var4)
```

[ ]:
```
# 22. Is the following an expression
x = 15
```

[ ]:
```
# 23. Is the following an expression
x**5
```

[ ]:
```
# 24. Is the following an expression

x + -100
```

[ ]:
```
# 25. Is the following an expression

a = x - 4
```

[ ]:
```
# 26. Is the following an expression

print(x)
```

[ ]:
```
# 27. What is the value of each of the following expressions when evaluated by␣
 ↪the Python interpreter?
print(3/2*4)
print(3/(2*4))
print((2*3)-4)
print(2**3+1)
print(2**(3+1))
print(2**3/4+2)
print("hello"*2)
print("Hello" + " " + "Dolly!")
```

[ ]:
```
# 28. What would go wrong here?
def calc_prod(x, y):
        return = x * y

print(calc_prod(777, 560))
```

[ ]:
```
# 29. What would go wrong here?
def calc_sum(x, y):
        return x + y

print(x, 'plus', y, 'is', calc_sum(777, 560))
```

[ ]:
```
# 30. What, if anything, would go wrong here?

4 > 2 and "4" > 2
```

[ ]:
```
# 31. What, if anything, would go wrong here?

4 > 2 or "4" > 2
```

[ ]:
```
# 32. What, if anything, would go wrong here?

"4" > 2  or 4 >
```

1.1 Solutions to coding assignments

2 Largest number

1. Write a function that uses a while loop to ask the user to enter many numbers, then returns the largest number entered. The prompt asking for a number should indicate that entering -1 will cause the loop to end and the loop should, in fact, end when the user enters -1.
2. The main body of the program (should follow the function definition) should call the function for testing purposes. Hint: it will probably be easier if you use more than one variable to help you keep track of the largest number entered, but you shouldn't need a lot of variables either.

[ ]:
```
# largest number
# example input: 3 1 5 10 -1

x=int(input("Enter a number. (Enter -1 to stop)"))
largestnumber=x
while x!=-1:
    x=int(input("Enter a number"))
    if x>largestnumber:
        largestnumber=x
print(largestnumber)
```

3 Leap years

A year is a leap year if it is divisible by 4, unless it is a century year (divisible by 100) that is not divisible by 400. Some examples:

- 2020 was a leap year because it is divisible by 4.
- 1900 was not a leap year, because it is divisible by 100, but not 400
- 2000 was a leap year, because it is divisible by 400

[ ]:
```
# Write a function that takes a year as a parameter and determines whether or␣
 ↪not the given year is a leap year.

def is_leap_year(year):
```

```
    """Determine if the given year is a leap year."""
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        print(year," is a leap year.")
    else:
        print(year," is not a leap year.")
```

3.1 (Optional) Add a while loop to the main body of the program that:

1. Asks the user to enter the year they want to check
2. Calls the function you created
3. Asks the user if they want to check another year (y/n)

[ ]:
```
# leap year - while loop
# example input: 1992 n

continue_checking = 'y'
while continue_checking == 'y':
    year = int(input("Enter the year you want to check: "))
    is_leap_year(year)
    continue_checking = input("Do you want to check another year? (y/n): ")
```

4 Rolling Dice

1. Write a function that takes two parameters, faces and num_dice that indicate how many sides a single die has and how many dice it should roll respectively.
   - random.randint may be helpful here.
2. Add two while loops that check the numbers the user enters:
   - The first loop should check the number of faces and should continue running until the user provides an integer larger than one
   - The second loop should check the number of dice the user will roll and should continue running until the user enters an integer larger than zero
   - These loops ensure that we have at least one side on each "die" and that we roll at least one die.
3. Call the function you wrote to roll the dice
4. Add another while loop that encapsulates the previous two loops as well as the call to the function.
   - This loop should ask the user if they want to roll another set of dice (y/n) and continue so long as the user indicates they want to coninue

[ ]:
```
# rolling dice
# example input: 6 1

import random
def roll_dice(faces, num_dice):
    """Roll dice and return the results."""
    for i in range(num_dice):
        roll_result=random.randint(1, faces)
```

```
    return roll_result

# Loop to ensure valid number of faces
faces = 0
while faces <= 1:
        faces = int(input("Enter the no. of faces on the die (greater than 1):␣
 ↪"))
# Loop to ensure valid number of dice
num_dice = 0
while num_dice <= 0:
        num_dice = int(input("Enter the no. of dice to roll (greater than 0):␣
 ↪"))
```

[ ]:
```
# rolling dice - continuous
# example input: y  n
continue_rolling = input("Do you want to roll a set of dice? (y/n): ")
while continue_rolling == 'y':
    # Loop to ensure valid number of faces
    faces = 0
    while faces <= 1:
        faces = int(input("Enter the no. of faces on the die (greater than 1):␣
 ↪"))
    # Loop to ensure valid number of dice
    num_dice = 0
    while num_dice <= 0:
        num_dice = int(input("Enter the no. of dice to roll (greater than 0):␣
 ↪"))

    # Roll the dice and display results
    results = roll_dice(faces, num_dice)
    print(f"You rolled: {results}")
    # Ask if the user wants to roll again
    continue_rolling = input("Do you want to roll another set of dice? (y/n): ")
```

5 Running for Congress

A person is eligible to be a US senator if they are at least 30 years old and have been a US citizen for at least 9 years. To be a US representative, you need to be at least 25 years old and have been a US citizen for at least 7 years. Write a function called eligible_to_run that takes two numbers indicating a person's age and years of citizenship, then prints a message stating their eligibility for the Senate and House.

The message should have one of the following forms:

- Congratulations! You are eligible to run for the House of Representatives and the Senate.
- You are eligible to run for the House of Representatives, but not for the Senate.
- Sorry! You are not eligible to run for Congress.

Make sure to test your implementation thoroughly. What are the different scenarios that should be tested? Show what you test by including a list of all function calls at the bottom of your file.

5.0.1 (Optional 1)

In addition to the citizenship and age requirements, an individual must be a resident of the state they are seeking to represent. Adjust the function you wrote to consider whether the person is a resident of the state or not.

[ ]:
```
# running for congress

def eligible_to_run(age, years_citizenship, is_resident=False):
    """Check eligibility for US Senate and House of Representatives."""
    if is_resident:
        #print("You are a resident of the state.")
        if  age >= 30 and years_citizenship >= 9:
            print("Congratulations! You are eligible to run for the House of␣
 ↪Representatives and the Senate.")
        elif age >= 25 and years_citizenship >= 7:
            print("You are eligible to run for the House of Representatives,␣
 ↪but not for the Senate.")
        else:
            print("Sorry! You are not eligible to run for Congress.")
    else:
        print("Sorry! You are not eligible to run for Congress.")
```

[ ]:
```
eligible_to_run(8,7,True)
eligible_to_run(100,7,True)
eligible_to_run(26,8,True)
eligible_to_run(50,50,False)
eligible_to_run(50,50,True)
```

5.0.2 (Optional 2)

In order to be President of the US, you need to be at least 35 years old, a natural born citizen, and have lived in the US for at least 14 years. Create a new function to check whether the person is eligible to become US president.

[ ]:
```
# president eligibility

def eligible_to_be_president(age, years_resident, is_natural_born_citizen ):
    """Check eligibility to become US President."""
    if age >= 35 and is_natural_born_citizen and years_resident >= 14:
        print("Congratulations! You are eligible to run for President of the␣
 ↪United States.")
    else:
        print("Sorry! You are not eligible to run for President.")
```

[ ]:
```
eligible_to_be_president(8,7,True)
eligible_to_be_president(100,7,True)
eligible_to_be_president(26,8,True)
eligible_to_be_president(50,50,False)
eligible_to_be_president(50,50,True)
```